

Netflix Movie Recommendation Engine

Project Summary & Code Walkthrough

Python

SVD

Collaborative Filtering

scikit-surprise

A full end-to-end machine learning pipeline that parses the Netflix Prize Dataset, applies collaborative filtering via SVD, and generates personalised movie recommendations.

01 | PROJECT OVERVIEW

This project builds a **Movie Recommendation Engine** modelled on Netflix's real-world recommendation infrastructure. Starting from raw, unstructured text data, it moves through every stage of a professional ML pipeline: custom parsing, data cleaning, exploratory analysis, noise reduction, model training, cross-validation, and personalised prediction.

The core algorithm is **Singular Value Decomposition (SVD)** — a matrix factorisation technique that uncovers hidden patterns in user behaviour to predict which unseen movies a given user is most likely to enjoy.

02 | PROBLEM STATEMENT

OTT platforms like Netflix and Amazon Prime handle millions of titles. Without intelligent recommendations, users face overwhelming choice — and churn. This project replicates the logic that sits behind the 'Top Picks for You' row by learning each user's taste from their rating history and ranking every unwatched movie by predicted personal score.

03 | DATASET AT A GLANCE

Cust_Id	Unique customer identifier (anonymised)
Rating	Star rating given by the customer — integer 1 to 5
Movie_Id	Unique movie identifier (parsed from raw file structure)
Year	Release year of the movie
Name	Movie title from the companion metadata file

Source: Netflix Prize Dataset (combined_data_1.txt + movie_titles.csv). The raw file is non-standard — Movie IDs are embedded inline as row markers rather than as a dedicated column.

04 | OBJECTIVES

- Identify the most popular and highly rated genres on the platform.
- Build a model that recommends the best-suited movies for any given user.
- Determine which genre categories attract the highest and lowest average ratings.

05 | STEP-BY-STEP WORKFLOW & DECISION RATIONALE

Every decision in this project was deliberate. Below is not just what was done, but the reasoning behind each engineering and data-science choice.

1 ■ Data Loading & Parsing

WHAT WAS DONE

Loaded `combined_data_1.txt` without headers, keeping only `Cust_Id` and `Rating` columns. Wrote a custom loop to extract Movie IDs from inline colon-separated markers and attached them as a new `Movie_Id` column. The date column was dropped at load time.

- The Netflix Prize file is not a standard CSV — Movie IDs appear as rows like '1:' followed by customer ratings for that movie. Without custom parsing, no rating can be matched to a movie.
- Dropping the date column early reduces memory overhead. For a rating-based recommendation engine, when a user watched something matters less than what they rated it.
- Manually assigning column names (`header=None`) was necessary because the raw file contains no header row.

2 ■ Data Preprocessing

WHAT WAS DONE

Removed NaN rows from the `Rating` column (these were movie ID markers, not real data). Converted `Cust_Id` from string/object type to integer. Computed dataset-wide statistics: total unique movies, total unique customers, and total number of ratings.

- NaN `Rating` rows were formatting artefacts of the raw file structure — keeping them would corrupt any aggregation or model training step.
- Converting `Cust_Id` to integer is required for ML models and comparison operations; leaving it as a string causes downstream errors.
- Computing summary statistics is a standard sanity check — it validates the dataset and gives a clear picture of its scale before any modelling begins.

3 ■ Exploratory Data Analysis (EDA)

WHAT WAS DONE

Visualised the distribution of star ratings (1 through 5) using a horizontal bar chart, showing the aggregate count for each rating level across the full dataset.

- Before building any model, understanding data shape and bias is essential. If most ratings were 5-stars, the model would skew everything highly — producing useless recommendations.
- The bar chart gives an immediate human-readable snapshot of overall user sentiment and helps identify whether any pre-processing adjustments are needed.

4 >■ Pre-filtering — Noise Reduction

WHAT WAS DONE

Two quantile-based filters were applied before training: movies rated by fewer than the 60th-percentile reviewer count were dropped; customers who had rated fewer than the 60th-percentile movie count were also removed.

- Movies with very few ratings are statistically unreliable — 2 or 3 ratings provide insufficient signal for the model to learn meaningful patterns.
- Inactive users don't provide enough behavioural data for the model to understand their preferences; including them adds noise without contributing useful patterns.
- The 60th percentile was chosen over a fixed number because it scales with the data — an adaptive threshold that removes the bottom 60% of low-engagement movies and users, retaining only the richest, most representative portion.

5 ■ Model Building — SVD

WHAT WAS DONE

Loaded movie_titles.csv with ISO-8859-1 encoding. Used scikit-surprise's SVD algorithm with a Reader object to format the data. Trained on the top 100,000 filtered rows for practical runtime.

- ISO-8859-1 encoding was required because movie titles contain special/accented characters that cause UTF-8 decoding errors.
- SVD was chosen over simpler methods (average ratings, content filters) because it is a collaborative filtering technique — it learns from the collective behaviour of all users, not just movie attributes.
- The Reader object specifies the rating scale (1–5) so SVD knows the value range during matrix decomposition.
- 100,000 rows was a deliberate performance trade-off — large enough for meaningful patterns but fast enough to run in a notebook environment.

6 ■ Model Evaluation

WHAT WAS DONE

Evaluated using 3-fold Cross-Validation with Root Mean Square Error (RMSE) as the performance metric.

- 3-fold CV was used instead of a single train/test split because it provides a more reliable performance estimate — the model is trained and tested across three different splits, reducing the chance of a lucky or unlucky split.
- RMSE directly measures the average error in predicted star ratings — an RMSE of 1.0 means predictions are off by ~1 star on average. It is the standard metric for regression-style recommendation problems.

7 ■ Generating Recommendations

WHAT WAS DONE

Selected a target user (Cust_Id: 1055714). Made a copy of the full movie catalogue, re-applied the pre-filter to exclude low-benchmark movies, computed an Estimate Score for every unseen movie using SVD predict(), sorted descending, and returned the Top 5 movies.

- Making a per-user copy of the catalogue avoids modifying the original df_title DataFrame — since an Estimate_Score column is added per user, working on a copy keeps the base data clean and reusable.
- Re-applying the pre-filter at output time ensures the engine never recommends a low-quality movie even if it wasn't seen by the user.
- Estimate Score is SVD's predicted rating for a user-movie pair — the higher the score, the more aligned that movie is with the user's history.
- Returning Top 5 gives a concise, actionable list — mirroring how real streaming platforms surface recommendations.

06 | WHY SVD WORKS FOR RECOMMENDATIONS

Singular Value Decomposition (SVD) decomposes the user-item rating matrix into three smaller matrices that together capture the latent relationships between users and movies. Conceptually, it asks: *'What hidden factors explain why certain users like certain movies?'*

These latent factors might represent genre preference, pacing, tone, or narrative style — none of which are explicitly labelled in the data, but which SVD discovers automatically from rating patterns alone.

Fills sparse data	Most users rate only a tiny fraction of all available movies. SVD intelligently fills in the missing values by learning from similar users and similar movies.
No content labels needed	SVD doesn't need genre tags or cast information — it learns purely from how users rate movies relative to each other.
Scalable	Once trained, predicting a score for any user-movie pair is a fast dot-product operation — no re-training required per user.
Proven	SVD was a core component of the winning entry in the original Netflix Prize competition — the same dataset this project uses.

07 | TECH STACK

pandas	Data loading, column engineering, filtering, and aggregation
numpy	Numerical array operations and benchmarking calculations
matplotlib	Bar chart visualisation of star-rating distribution
seaborn	Statistical visualisation layer over matplotlib
scikit-surprise	SVD collaborative filtering model and cross-validation

08 | KEY FEATURES

- ✓ End-to-End Pipeline — raw data parsing through to final recommendations
- ✓ Custom Data Parser — handles the non-standard inline Movie ID format
- ✓ Percentile-Based Pre-filtering — adaptive noise removal at the 60th quantile
- ✓ SVD Collaborative Filtering — matrix factorisation uncovers latent preferences
- ✓ 3-Fold Cross-Validated Model — robust RMSE-based performance evaluation
- ✓ Scalable Output — easily extendable to any customer ID in the dataset
- ✓ Visual EDA — rating distribution chart for instant data comprehension

09 | FUTURE IMPROVEMENTS

- Genre-level filtering for category-specific recommendation lanes
- Batch pipeline to generate recommendations for all users simultaneously
- Experiment with ALS and NMF as alternative matrix factorisation methods

- Interactive web interface (Flask / Streamlit) for live recommendations
- Integration with all four combined Netflix Prize data files

Built using the Netflix Prize Dataset · scikit-surprise · Python 3.x

For educational and research purposes.